



ThreadCraft Academy

Stop guessing what `await`, `lock` and the thread pool do — watch them work, then prove it in the editor.

User Manual · 100 lessons across five chapters · Updated 30 July 2026

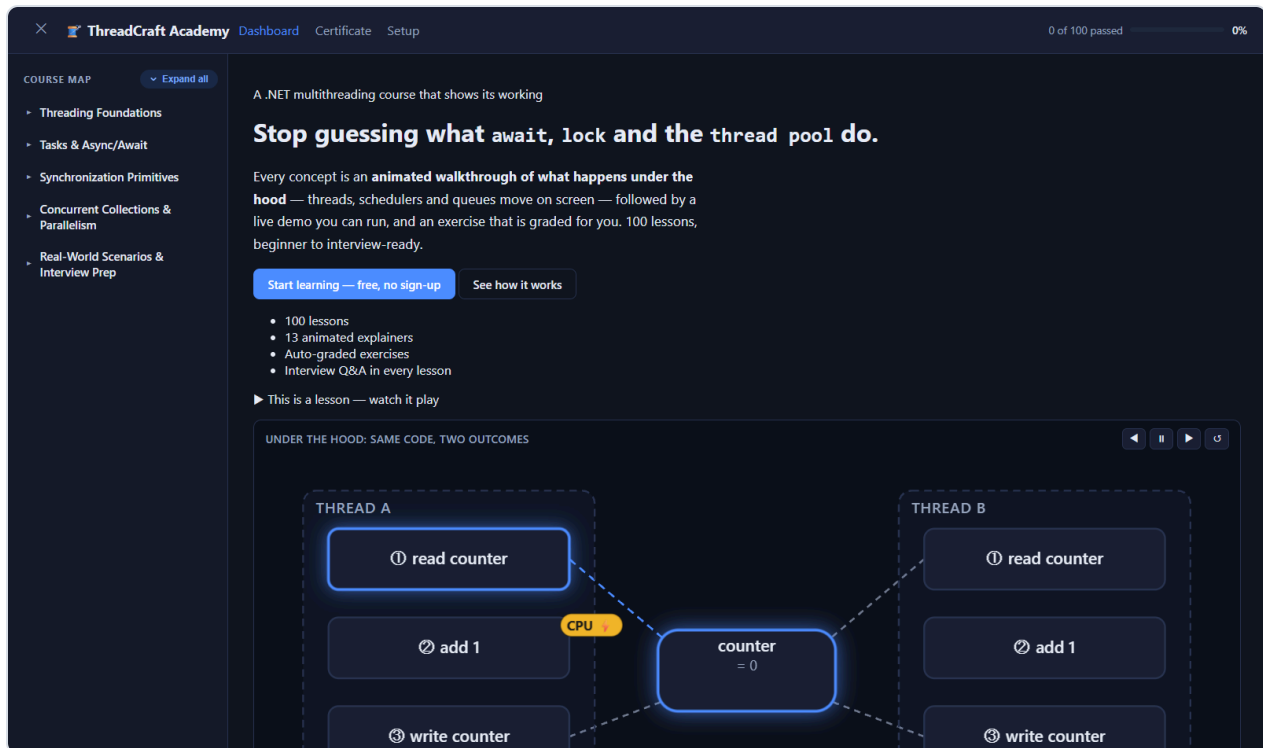
Contents

1. What ThreadCraft Academy is
2. Getting started
3. Finding your way around
4. How every lesson works
5. The animated explainer
6. The live sandbox demo
7. Exercises and auto-grading
8. Hints, solutions and the AI Coach
9. Tracking progress and the certificate
10. The Setup diagnostics page
11. The curriculum
12. Troubleshooting

1. What ThreadCraft Academy is

ThreadCraft Academy is an interactive course in .NET multithreading and concurrency. It exists because threading is the topic where reading rarely produces understanding: the behaviour is invisible, timing-dependent, and often counter-intuitive.

So every lesson shows you the behaviour three ways. You **watch** an animated explainer step through what the threads are actually doing, you **run** a live sandbox demo and see the real output, and then you **prove** you understood it by writing code that is automatically checked.



The ThreadCraft Academy home page.

One hundred lessons take you from creating your first thread to the concurrency patterns that come up in senior .NET interviews.

2. Getting started

Use the hosted version

The course runs in your browser at `threadcraft-academy.azurewebsites.net`. No sign-up, and your progress is kept for you as you work.

Or run it locally

You need the .NET 8 SDK or newer, and PowerShell.

```
git clone https://github.com/johancs/threadvana threadcraft
cd threadcraft
powershell -ExecutionPolicy Bypass -File setup.ps1
```

The setup script checks your SDK, builds the solution and opens the app. Useful flags:

Flag	What it does
(none)	Build and launch in your browser
-Clean	Clean rebuild from scratch, then launch
-NoBrowser	Start the server without opening a browser
-Publish	Build for production to <code>./publish</code>

3. Finding your way around

The screenshot shows the ThreadCraft Academy interface. At the top, there are navigation links: **ThreadCraft Academy**, **Dashboard**, **Certificate**, and **Setup**. On the right, it shows progress: "0 of 100 passed" and "0%".

The left sidebar is titled "COURSE MAP" and has a "Collapse all" button. It lists five chapters under "Threading Foundations":

- What is a Thread?
- Why Threads at All?
- Giving Your Thread Some Data
- Join - Waiting for a Worker to Finish
- Foreground vs Background Threads
- Naming Threads and Watching Their State
- Pausing a Thread - Sleep and Yield
- The Cost of a Thread
- Meet the Thread Pool - Borrow Workers Instead of Hiring
- Watch the Pool Reuse Its Workers
- Tuning the Thread Pool - Min Threads and the Slow Ramp-Up
- Exceptions Don't Cross Threads
- Thread-Local Storage - Every Thread Gets Its Own Copy
- Telling a Thread to Stop - Cooperative Cancellation
- Timers - Running Code on a Schedule
- Choosing the Right Tool - Sync, Async, or Parallel?
- Debugging Multithreaded Code - Survival Tactics

The main content area shows a lesson overview for "A .NET multithreading course that shows its working". The title is "Stop guessing what await, lock and the thread pool do." Below the title, it says: "Every concept is an **animated walkthrough of what happens under the hood** — threads, schedulers and queues move on screen — followed by a live demo you can run, and an exercise that is graded for you. 100 lessons, beginner to interview-ready."

There are two buttons: "Start learning — free, no sign-up" and "See how it works". Below these, a list of features is shown:

- 100 lessons
- 13 animated explainers
- Auto-graded exercises
- Interview Q&A in every lesson

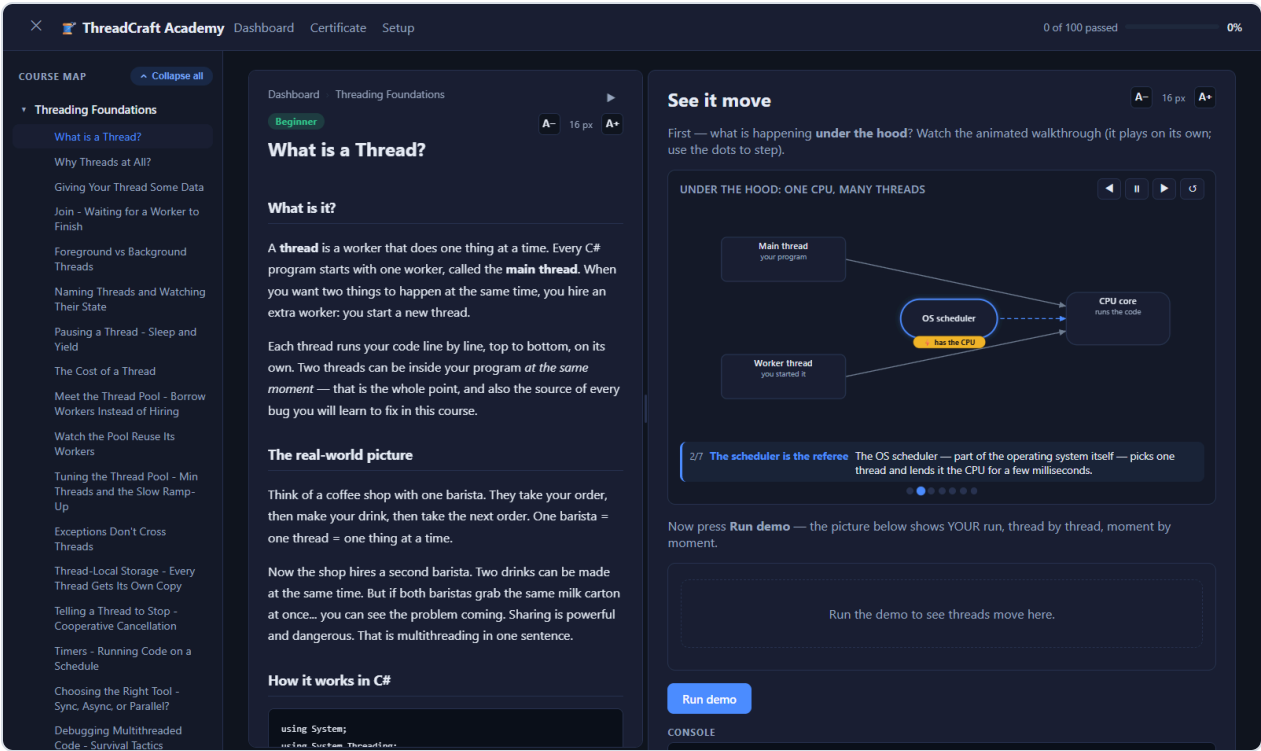
A play button icon indicates "This is a lesson — watch it play". Below this is a diagram titled "UNDER THE HOOD: SAME CODE, TWO OUTCOMES". The diagram shows two threads, "THREAD A" and "THREAD B", each with a sequence of actions: "1 read counter", "2 add 1", and "3 write counter". These actions are connected to a central "counter = 0" box, which is also labeled "CPU".

The sidebar holds all five chapters; expanding one lists its twenty lessons.

The left sidebar is the curriculum. Five chapters, each holding twenty lessons, expand to show their contents; **Expand all** opens every chapter at once. The sidebar can be hidden entirely when you want the full width for reading or coding.

Three links sit at the top: **Dashboard** for your progress, **Certificate** for your completion record, and **Setup** for diagnostics.

4. How every lesson works



A lesson page. Each follows the same predictable structure.

Every lesson is laid out the same way, so you always know where you are:

Section	What it gives you
What is it?	The concept in plain language.
The real-world picture	An analogy that makes the mechanism intuitive.
How it works in C#	The actual API and idioms, with code.
See it move	The animated explainer and the runnable demo.
Watch out	The traps — the bugs this feature actually causes in production.
Key takeaways	What to remember.
Your turn	The graded exercise.
Interview corner	How the topic tends to be asked about.
Ask the coach	The AI tutor, in the context of this lesson.

5. The animated explainer

The screenshot shows the ThreadCraft Academy interface. On the left is a 'COURSE MAP' with a list of topics under 'Threading Foundations'. The main area is titled 'See it move' and contains text explaining thread execution, a 'Watch out' section with common pitfalls, and 'Key takeaways'. To the right of the text is a diagram titled 'UNDER THE HOOD: ONE CPU, MANY THREADS' showing a 'Main thread' and a 'Worker thread' both connected to an 'OS scheduler', which in turn connects to a 'CPU core'. Below the diagram is a 'Run demo' button and a 'CONSOLE' section.

ThreadCraft Academy Dashboard Certificate Setup 0 of 100 passed 0%

COURSE MAP Collapse all

- Threading Foundations
 - What is a Thread?
 - Why Threads at All?
 - Giving Your Thread Some Data
 - Join - Waiting for a Worker to Finish
 - Foreground vs Background Threads
 - Naming Threads and Watching Their State
 - Pausing a Thread - Sleep and Yield
 - The Cost of a Thread
 - Meet the Thread Pool - Borrow Workers Instead of Hiring
 - Watch the Pool Reuse Its Workers
 - Tuning the Thread Pool - Min Threads and the Slow Ramp-Up
 - Exceptions Don't Cross Threads
 - Thread-Local Storage - Every Thread Gets Its Own Copy
 - Telling a Thread to Stop - Cooperative Cancellation
 - Timers - Running Code on a Schedule
 - Choosing the Right Tool - Sync, Async, or Parallel?
 - Debugging Multithreaded Code - Survival Tactics

See it move A- 16 px A+

First — what is happening **under the hood**? Watch the animated walkthrough (it plays on its own; use the dots to step).

UNDER THE HOOD: ONE CPU, MANY THREADS

Main thread
your program

Worker thread
you started it

OS scheduler
has the CPU

CPU core
runs this code

2/7 **The scheduler is the referee** The OS scheduler — part of the operating system itself — picks one thread and lends it the CPU for a few milliseconds.

Now press **Run demo** — the picture below shows YOUR run, thread by thread, moment by moment.

Run the demo to see threads move here.

Run demo

CONSOLE

console output will appear here when you run the demo

The step-through animation: play, pause, step back and forth, or jump to any numbered step.

The explainer animates what the threads are doing, one step at a time. You can play it, pause it, step backwards and forwards, restart it, or jump straight to any numbered step.

Stepping matters more than playing. Threading bugs come from a specific interleaving, and being able to stop on the exact step where two threads touch the same variable is what makes a race condition finally make sense.

6. The live sandbox demo

The screenshot displays the ThreadCraft Academy interface. On the left is a 'COURSE MAP' sidebar with a list of topics under 'Threading Foundations', including 'What is a Thread?', 'Why Threads at All?', 'Giving Your Thread Some Data', 'Join - Waiting for a Worker to Finish', 'Foreground vs Background Threads', 'Naming Threads and Watching Their State', 'Pausing a Thread - Sleep and Yield', 'The Cost of a Thread', 'Meet the Thread Pool - Borrow Workers Instead of Hiring', 'Watch the Pool Reuse Its Workers', 'Tuning the Thread Pool - Min Threads and the Slow Ramp-Up', 'Exceptions Don't Cross Threads', 'Thread-Local Storage - Every Thread Gets Its Own Copy', 'Telling a Thread to Stop - Cooperative Cancellation', 'Timers - Running Code on a Schedule', 'Choosing the Right Tool - Sync, Async, or Parallel?', and 'Debugging Multithreaded Code - Survival Tactics'. The main content area is titled 'See it move' and contains a 'Watch out' section with bullet points about thread execution, a 'Key takeaways' section with a list of facts about threads, and a 'Run demo' button. To the right of the main content is a diagram titled 'UNDER THE HOOD: ONE CPU, MANY THREADS' showing the interaction between a 'Main thread (your program)', a 'Worker thread (you started it)', an 'OS scheduler', and a 'CPU core (runs this code)'. A callout box explains a 'Context switch' where the scheduler freezes the main thread to run the worker. Below the diagram is a 'Run the demo to see threads move here.' button and a 'CONSOLE' section showing the output of the program.

Run demo executes real code and shows you its actual output.

Run demo executes the lesson's code and shows the real output. This is not a recording — for lessons about non-determinism, running it repeatedly gives different results, which is precisely the point being taught.

7. Exercises and auto-grading

The screenshot shows the ThreadCraft Academy interface. On the left is a 'COURSE MAP' with a list of topics under 'Threading Foundations'. The main content area is divided into three sections: 'See it move' with a timeline diagram and text, 'Watch out' with a list of common mistakes, and 'Key takeaways' with a list of important points. On the right, the 'Your turn' section contains instructions for an exercise, a code editor with a C# solution, and buttons for 'Check my code', 'Show a hint (1/3)', and 'Show solution'. The interface also shows a progress bar at the top right indicating '0 of 100 passed' and '0%'.

See it move

Press **Run demo**. The timeline shows one swimlane per thread. Watch how the main thread and the worker thread overlap in time — both are running at once. Then look at the `Join` point: the main lane goes grey (waiting) until the worker lane ends.

Watch out

- You might think the new thread's lines always print after the main thread's lines. They don't. Once you call `Start()`, the operating system decides who runs when.
- You might forget `Join()`. If the main thread reaches the end of the program, your worker can be cut off mid-work.
- You might create threads in a loop for lots of small tasks. Threads are expensive; later lessons (thread pool!) show the cheaper way.

Key takeaways

- A thread is a worker that runs code one line at a time.
- Every program starts with one main thread; `new Thread(...)` + `Start()` adds more.
- Threads run **at the same time**, in an order you cannot predict.
- `Join()` waits for a thread to finish.
- Threads share the same memory — great for teamwork, dangerous for shared data.

Your turn

Start your very first thread.

Inside `Solution.Run()`:

- Create a `new Thread(...)`. Inside the thread's work, store the id of the thread it is running on: `Solution.WorkerThreadId = Environment.CurrentManagedThreadId;`
- Call `Start()` on your thread.
- Call `Join()` so `Run()` only returns after your thread has finished.

We check two things: that `WorkerThreadId` got set at all, and that it is a *different* id than the thread that called `Run()` — proof that a new worker really did the job.

```
1 using System;
2 using System.Threading;
3
4 public static class Solution
5 {
6     // The checker reads this after Run() returns.
7     // Your new thread must store its own thread id here.
8     public static int WorkerThreadId = 0;
9
10    public static void Run()
11    {
12        // TODO: 1. Create a new Thread. Inside its work, set
13        //         WorkerThreadId = Environment.CurrentManagedThreadId;
14        //         2. Start the thread.
15        //         3. Join it, so Run() waits until the work is done.
16    }
17 }
18
```

Check my code 2 attempts so far

Show a hint (1/3)

Show solution

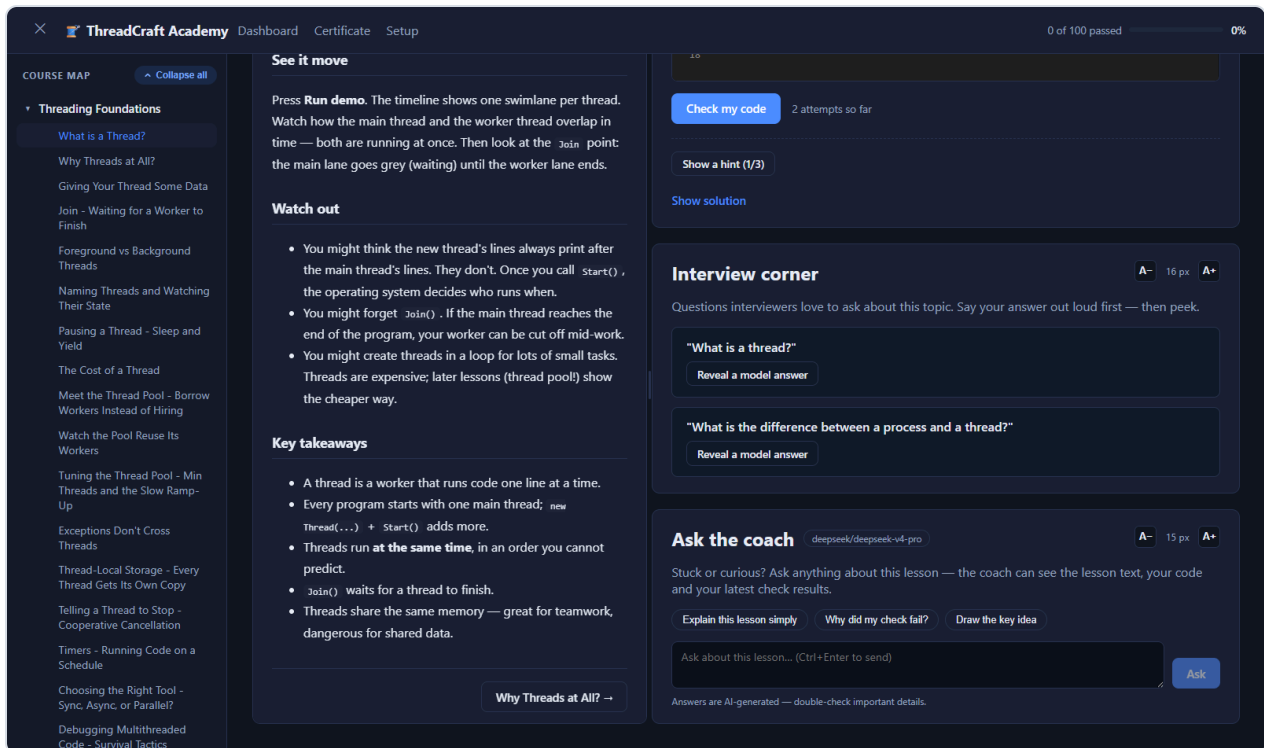
Your turn: write the code, then Check my code grades it automatically.

Each lesson ends with an exercise in an editor. **Check my code** compiles and evaluates your solution and tells you whether it satisfies the requirement — you are graded on behaviour, not on matching a model answer.

8. Hints, solutions and the AI Coach

If you are stuck, help is graduated so you can take only as much as you need:

- Show a hint** — up to three per lesson, revealed one at a time.
- Show solution / Reveal a model answer** — the full worked answer.



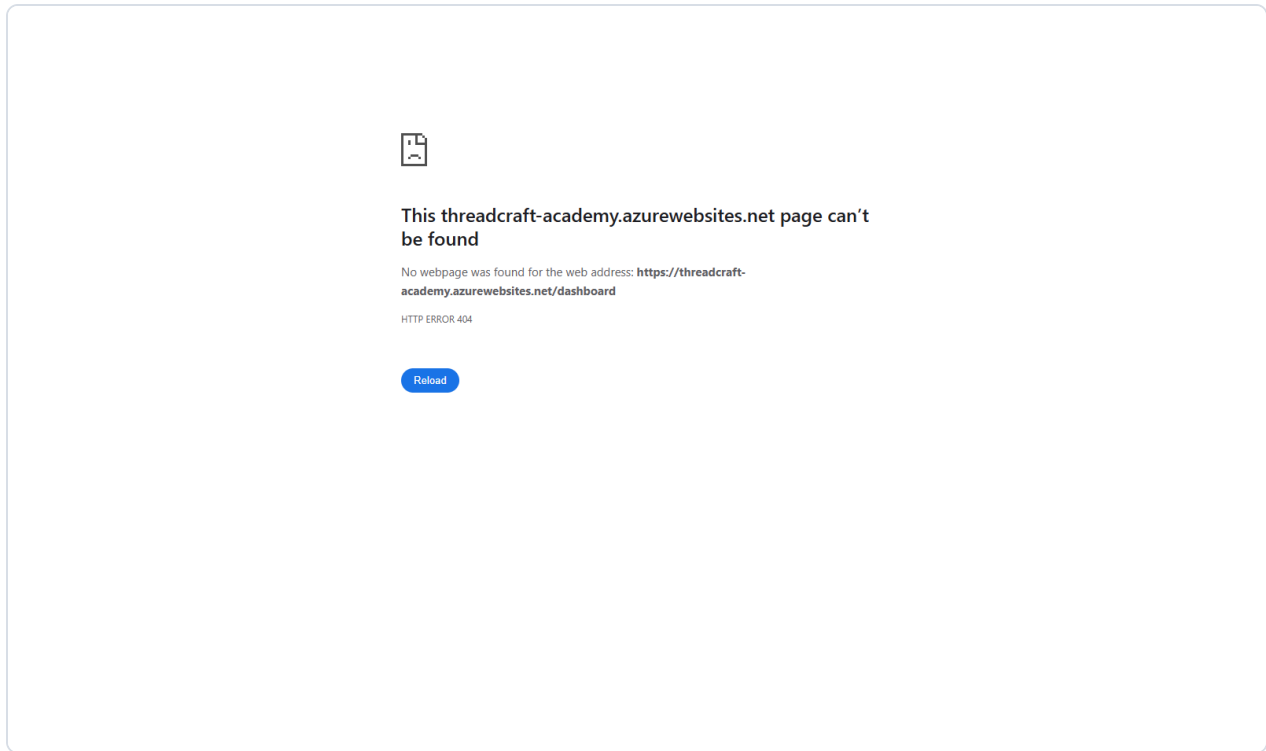
The AI Coach, with one-click prompts tuned to where learners usually get stuck.

The **AI Coach** answers questions about the lesson you are on. It offers one-click prompts for the situations that come up most:

Prompt	Use it when
<i>Explain this lesson simply</i>	The concept has not landed yet.
<i>Why did my check fail?</i>	Your exercise was rejected and you want to know why.
<i>Draw the key idea</i>	You would rather see the mechanism than read about it.

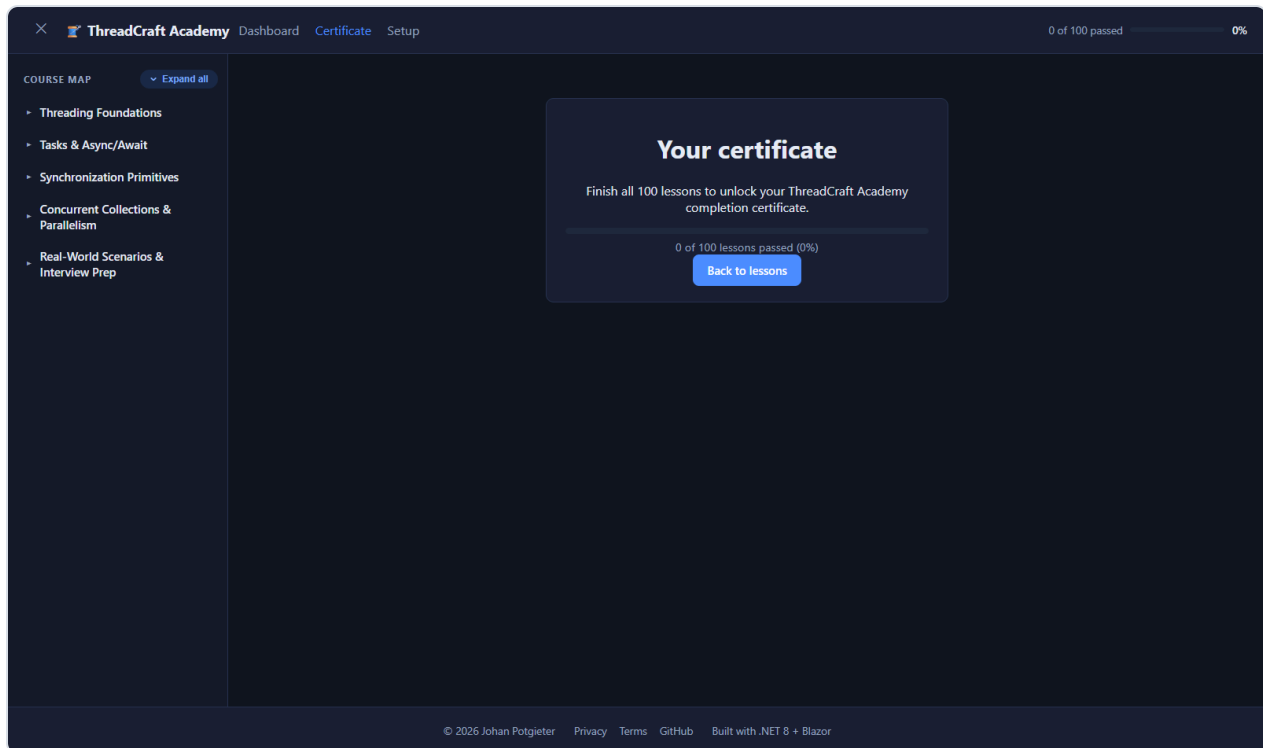
The Coach is optional and needs an OpenRouter API key. Without one, the panel shows setup instructions and everything else works normally. Set `OPENROUTER_API_KEY`, or add an `Assistant` section to `appsettings.json`.

9. Tracking progress and the certificate



The Dashboard: how far you are through each chapter and the course.

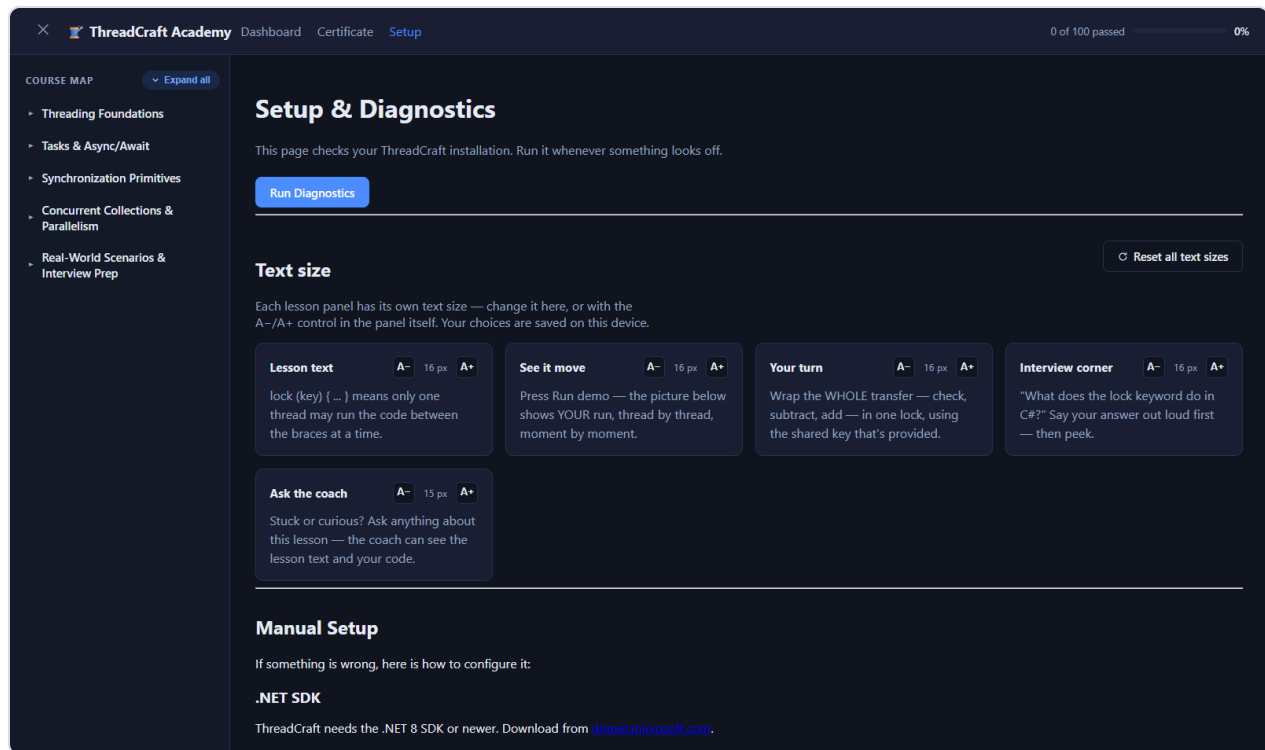
The **Dashboard** shows how much of each chapter you have completed and your progress overall, so you can pick up where you left off.



The Certificate page, which records your completion of the course.

The **Certificate** page records your completion once you have worked through the curriculum.

10. The Setup diagnostics page



Setup runs diagnostics and tells you which parts of the environment are working.

Setup verifies your environment and reports what is and is not working — the first place to look if demos will not run or the Coach is silent.

11. The curriculum

One hundred lessons in five chapters, each building on the last.

Chapter	What it covers
1. Threading Foundations	Creating threads, passing data, <code>Join</code> , foreground versus background, the thread pool and how it reuses workers, the true cost of a thread, thread-local storage, cooperative cancellation, timers, and a first look at race conditions and starvation.
2. Tasks & Async/Await	<code>Task</code> as a promise of a future result, what <code>async</code> and <code>await</code> really do, and the machinery underneath.
3. Synchronization Primitives	<code>lock</code> , semaphores and the rest of the toolbox — what each one guarantees and what it costs.
4. Concurrent Collections & Parallelism	Thread-safe collections and parallel execution.
5. Real-World Scenarios & Interview Prep	Putting it together, and the questions interviewers actually ask.

Chapters end in a capstone that combines what you have learned — chapter one closes with a mini download manager.

12. Troubleshooting

Symptom	What to do
Demos will not run	Open Setup ; it reports which parts of the environment are healthy.
The AI Coach shows setup instructions instead of answering	No API key is configured. See section 8 — everything else still works.
The hosted site is slow to load first time	It runs on a free App Service tier, which sleeps when idle. The first request wakes it.
A demo gives a different result each run	That is usually the lesson. Non-determinism is the behaviour being demonstrated.
Local build fails	Check you have the .NET 8 SDK or newer, then try <code>setup.ps1 -Clean</code> .

